

jev-decision-service 0.1.3 · 六项缺陷定位与修复

真机环境：NVIDIA DGX Spark (GB10) + ollama + Qwen3-8B | 方法：逐字节 diff + 单元测试 + 真机端到端 + 缺陷复现对照 | 2026-09-24

0 结论

在真机上把 0.1.3 完整跑了一遍，定位并修复了 6 项缺陷，补丁覆盖 3 个文件、7 处修改（幂等，可重复执行）。其中两项属于安全机制失效级别——而且都是在「启用了原本更好的功能」之后才触发的。

6 项

定位的缺陷

7 处

补丁修改点

0/3 → 3/3

opensource 路径成功率

29

单元测试全绿

同时完成了哨兵决策头与 jev 框架的桥接：用 jev 的 QuestionProfile 定义工业版问题集，把哨兵决策头作为 StructuredHeadProvider 的 evaluator 接入，端到端跑通「设备状态 → 决策头 → 类型化决策 → 策略层路由」。

1 缺陷清单

#	缺陷	严重度	证据
1	opensource 路径提示词与解析器键名不匹配	阻断级	真机 0/3 成功；_parse_jev_answer 在 0.1.0 与 0.1.3 中逐字节相同（2574 字节）
2	模型输出的概率和不严格为 1	阻断级	ProviderError: choice probabilities do not sum to one（校验只容忍 0.01 偏差）
3	score 越界（4 档被模型理解成 1~4）	阻断级	ProviderError: score is out of range
4	ollama 默认 128 token 截断多问 JSON	阻断级	形状校验不通过；payload 未设置 max_tokens
5	启用校准后「紧急度保护」静默失效	★ 严重	severity=3 时：calibrated=False → human_review；calibrated=True → answer_from_context（保护被绕过）
6	「稀有类」告警是静态全局的，声明即永久转人工	★ 严重	同一台健康设备：未声明 → answer_from_context；声明了 → human_review + blocked

缺陷 1~4 是「跑不起来」，缺陷 5~6 是「跑起来了但保护失效」——后者更危险，因为它们不报错、不崩溃，系统看起来一切正常，但高危报警被当作普通消息回复、诚实标注数据局限反而让系统整体不可用。

2 缺陷 1~4：opensource 路径不可用

版本	第 1 次	第 2 次	第 3 次	成功率
0.1.3 原版	失败	失败	失败	0 / 3
0.1.3 + 补丁	通过 13.3 s	通过 13.2 s	通过 13.2 s	3 / 3

根因是提示词要和解析器认的不是一回事：解析器只接受 choice / score / noul 三个原生键名，而 system prompt 只说「using the declared choice options, numeric score levels, or noul yes probability」，从未要求把值放在这些键上。模型于是输出 "value" 键，解析器找不到，整单请求以 retryable=False 硬失败。

补丁修改点	作用
重写 system prompt	明确列出三种形状与键名、给示例、显式禁止 value 键
payload 增加 max_tokens	默认 1024，避免 ollama 默认 128 token 截断

补丁修改点	作用
dataclass 增加 max_tokens 字段	不同后端可调参
新增 _normalize_raw_answers()	兜住 value 键名 / 概率和不为 1 / score 越界 / 概率非数值四类瑕疵
在 _validate_answer_set 前调用清洗函数	只做容错，不改校验器的合法性判断

建议：这几类格式错误目前全部标为 retryable=False，意味着模型任何一处输出瑕疵都会让整单请求硬失败，而不会降级到 rules provider。建议改为 retryable=True，让 ResilientProvider 兜底。

3 缺陷 5：启用校准，紧急度保护反而失效（严重）

calibration.py 的 apply_profile 在多元分支里**无条件**把 value 改写成概率最高的标签名：

类型	value 本应是	被改写成
choice	选项名	选项名（正确）
score	数值（如 3.0）	标签名（如 '立即'）← 错误

而 policy.py 判断紧急度用的是：

代码	后果
tension = _find(predictions, "risk_level") ... isinstance(tension.value, (float, int)) and float(tension.value) >= urgent_score	value 变成字符串后 isinstance 判断为 False，整个「高危/紧急」分支被静默跳过

配置	value 实际类型	policy 动作	紧急度保护
calibrated=False	3.0 (float)	human_review	生效 ✓
calibrated=True	3.0 (float)	human_review	生效 ✓（修复后）
calibrated=True（修复前）	'立即' (str)	answer_from_context	失效 ✕

修复方式：只有 choice 类型才把 value 设为标签名；score 类型保持数值（取概率最高档位的下标）。修复后实测 calibrated=True 与 False 行为一致，紧急度保护恢复生效。

4 缺陷 6：越诚实，系统越不可用（严重）

apply_profile 只要发现 profile 声明了任意样本数低于阈值的类，就无条件往 warnings 里塞 rare_classes（与本次输入无关）：

情况	warnings	policy 动作	是否拦截
未声明稀有类（健康设备）	['label_provenance:derived_rule']	answer_from_context	否
声明了稀有类（健康设备）	['label_provenance:derived_rule', 'rare_classes:RNF,TWF']	human_review	是
声明了稀有类（修复后）	同上	answer_from_context	否（仅命中时才拦）

这形成一个自相矛盾的激励：越诚实地标注「我们某个类样本不足」，系统就越不可用——所有请求被永久转人工，自动化率归零。修复方式：改为仅当本次预测确实命中稀有类时才拦截。

5 补丁与用法

文件	修改点数	内容
provider.py	5	提示词键名、max_tokens、输出清洗函数与调用
calibration.py	1	score 类型的 value 保持数值
policy.py	1	稀有类告警改为「命中才拦截」

python3 patch_jev_service.py /path/to/package_root (包根目录含 src/ , 也可直接指向单个 .py 文件) ; 加 --check 只检测已应用情况。补丁幂等 , 重复执行不会重复插入。应用后 29 个单元测试全绿。

6 附带成果：哨兵决策头已接进 jev 框架

用 jev 的 QuestionProfile 定义了工业维护版问题集 , 并沿用 policy 认识的问题 id (risk_level / needs_evidence / needs_clarification / allows_side_effect / next_action) ——这样不需要改 policy 一行代码 , 它的五条保护全部继续生效。

场景	risk_level	next_action	最终动作
明显故障 (磨损 243min、温升 13.5K、扭矩 62.3Nm)	立即	human_review	retrieve_evidence (先取证据)
健康设备 (磨损 5min、温升 8.1K)	计划	answer_from_context	answer_from_context
边界 (磨损 130min、温升 9.0K)	尽快	answer_from_context	answer_from_context
过载 (2200rpm、扭矩 68Nm)	尽快	ask_clarification	human_review

桥接脚本 sentinel_jev_bridge.py 在节点上 4 秒完成三个决策头训练 (GPU) , 端到端跑通。这意味着团队的两条线可以合并 : 队友的框架 / 校准 / 策略层 + 哨兵的 GPU 决策头与实测数据。

补丁 : patch_jev_service.py | 桥接 : sentinel_jev_bridge.py | 真机脚本 : jev_real_test.py · calib_check.py | 全部结论来自 gx10-902c 实测